

Manual AWS — EC2 + Lambda + EventBridge (Issue #355)

Estado:  **ejecutado y funcionando** (corrida real 2026-09-17 — Lambda `chamba-matching-daily-refresh` y el schedule `chamba-matching-daily-refresh` creados y verificados). Lo que sigue es la referencia para volver a hacerlo en otro entorno (staging real, producción) — ver §"Troubleshooting real" en el paso 5 por los problemas que sí aparecieron y cómo se resolvieron.

Comandos para correr **desde AWS CloudShell**, en orden, copiando y pegando cada bloque. Pensado para ser reproducible: todos los IDs se derivan de la EC2 real ya existente (nunca se hardcodean), o se generan en el momento.

Decisiones ya tomadas para este manual (avisame si alguna no es la que querés): - **Acceso a la EC2 nueva: AWS Systems Manager (SSM Session Manager)** — sin key pair, sin IP pública obligatoria, sin puerto 22 abierto. Si preferís SSH tradicional, este manual cambia bastante (hay que sumar key pair + regla de ingreso 22 + decidir IP pública) — avisame y lo rehago. -

Alcance: solo el refresh diario (EventBridge → Lambda → EC2 start/stop). El disparo on-demand para usuario nuevo queda para un manual aparte (ver `README.md` de esta carpeta). - **Patrón de apagado:** la propia EC2 se apaga sola al terminar el refresh (`shutdown -h now` al final del script) — la Lambda dispara y no espera, evitando el límite de 15 minutos de ejecución de Lambda.

Quién corre qué — división real que terminamos usando (por los permisos de la cuenta real): el perfil de desarrollo (sin acceso a IAM) hace la parte de **descubrimiento** (pasos 1-3: VPC, subnet, security groups) y las verificaciones posteriores por SSM; **el administrador** corre todo lo que necesita `iam:PassRole` (pasos 4, 7, 9-11) — en la práctica eso significó que el admin creó **los 3 roles IAM juntos, de una sola vez, antes de lanzar nada** (por eso las políticas de los pasos 9 y 11 están acotadas por patrón de nombre — `instance/*`, `function:${PROJECT_NAME}-*` — en vez de por ARN exacto: todavía no existe la EC2/Lambda real al momento de crear esos roles).

0. Variables (ajustar antes de empezar)

```
export AWS_REGION="eu-central-1"
export PROJECT_NAME="chamba-matching"

# El instance-id de la EC2 que YA EXISTE (la del backend real) — pegalo acá.
# Si no lo sabés de memoria: aws ec2 describe-instances --region "$AWS_REGION" \
#   --query 'Reservations[].Instances[].[InstanceId,Tags[?Key==`Name`].Value][0],State.Name]' --output
table
export BACKEND_EC2_ID="i-XXXXXXXXXXXXXXXXX"

# Puerto real de Postgres en esa EC2 (5432 es el default de Postgres — confirmar
# si difiere; dev-infra local usa 5434 solo por un remapeo local, no es el real).
export BACKEND_DB_PORT="5432"

export INSTANCE_TYPE="t3.medium"
```

1. Descubrir VPC, subnet y security group de la EC2 existente

No se hardcodea nada — se saca todo de la instancia real:

```

export VPC_ID=$(aws ec2 describe-instances --region "$AWS_REGION" \
  --instance-ids "$BACKEND_EC2_ID" \
  --query 'Reservations[0].Instances[0].VpcId' --output text)

export SUBNET_ID=$(aws ec2 describe-instances --region "$AWS_REGION" \
  --instance-ids "$BACKEND_EC2_ID" \
  --query 'Reservations[0].Instances[0].SubnetId' --output text)

export BACKEND_SG_ID=$(aws ec2 describe-instances --region "$AWS_REGION" \
  --instance-ids "$BACKEND_EC2_ID" \
  --query 'Reservations[0].Instances[0].SecurityGroups[0].GroupId' --output text)

echo "VPC: $VPC_ID | Subnet: $SUBNET_ID | SG backend: $BACKEND_SG_ID"

```

Chequear si la subnet tiene salida a internet (decide si hace falta `--associate-public-ip-address` al lanzar la EC2 nueva en el paso 7). Ojo: si la subnet no tiene una route table asociada explícitamente (caso común), filtrar por `association.subnet-id` da **vacío** aunque sí tenga salida — hay que buscar la *main route table* de la VPC, que es la que aplica por default:

```

aws ec2 describe-route-tables --region "$AWS_REGION" \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=association.main,Values=true" \
  --query 'RouteTables[].Routes[].[DestinationCidrBlock,GatewayId,NatGatewayId]' \
  --output table

```

Si aparece una fila `0.0.0.0/0 -> igw-xxxxx` (Internet Gateway): subnet pública → `--associate-public-ip-address` sí hace falta (verificado en esta cuenta: subnet `subnet-0d7d3e49638abf187`, VPC `172.31.0.0/16`, con ruta a `igw-0db9bb160260ddddf` — es la VPC default de la cuenta). Si en cambio aparece un `NatGatewayId` (`nat-xxxxx`) sin ruta a un `igw-`: subnet privada con NAT — la EC2 no necesita IP pública, sacar ese flag del paso 7.

2. Crear el security group NUEVO (para la EC2 de ingesta/matching)

Sin reglas de entrada — con SSM no hace falta abrir ningún puerto entrante. La salida (scraping, pull de imágenes Docker, descarga del modelo BGE-M3) usa la regla de egress "allow all" que trae por default cualquier security group nuevo.

```

export NEW_SG_ID=$(aws ec2 create-security-group --region "$AWS_REGION" \
  --group-name "${PROJECT_NAME}-sg" \
  --description "Issue #355 - EC2 de ingesta/matching (pgvector)" \
  --vpc-id "$VPC_ID" \
  --query 'GroupId' --output text)

echo "SG nuevo: $NEW_SG_ID"

```

3. Abrir Postgres en el SG del backend EXISTENTE, solo desde el SG nuevo

⚠ **Este es el único paso que modifica infraestructura ya existente** — agrega una regla al security group del backend real, no lo reemplaza ni le saca nada.

```
aws ec2 authorize-security-group-ingress --region "$AWS_REGION" \
  --group-id "$BACKEND_SG_ID" \
  --protocol tcp --port "$BACKEND_DB_PORT" \
  --source-group "$NEW_SG_ID"
```

Verificar que quedó bien (y que no se abrió nada de más):

```
aws ec2 describe-security-groups --region "$AWS_REGION" --group-ids "$BACKEND_SG_ID" \
  --query 'SecurityGroups[0].IpPermissions'
```

4. Rol IAM + instance profile para la EC2 nueva (habilita SSM)

```
cat > /tmp/ec2-trust-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{"Effect": "Allow", "Principal": {"Service": "ec2.amazonaws.com"}, "Action":
"sts:AssumeRole"}]
}
EOF

aws iam create-role --role-name "${PROJECT_NAME}-ec2-role" \
  --assume-role-policy-document file:///tmp/ec2-trust-policy.json

aws iam attach-role-policy --role-name "${PROJECT_NAME}-ec2-role" \
  --policy-arn arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore

aws iam create-instance-profile --instance-profile-name "${PROJECT_NAME}-ec2-profile"

aws iam add-role-to-instance-profile \
  --instance-profile-name "${PROJECT_NAME}-ec2-profile" \
  --role-name "${PROJECT_NAME}-ec2-role"

# Dar unos segundos - IAM es eventually-consistent, el profile puede tardar
# en estar disponible para run-instances si se usa inmediatamente después.
sleep 15
```

5. AMI de Ubuntu 26.04 LTS más reciente (sin hardcodear un AMI ID que se pudra)

Canonical publica la AMI vigente vía SSM Parameter Store — siempre trae la última:

```
export AMI_ID=$(aws ssm get-parameters --region "$AWS_REGION" \
  --names /aws/service/canonical/ubuntu/server/26.04/stable/current/amd64/hvm/ebs-gp3/ami-id \
  --query 'Parameters[0].Value' --output text)

echo "AMI: $AMI_ID"
```

⚠ Troubleshooting real (pasó en la corrida del 2026-09-17)

Nunca cambies `$AWS_REGION` a mitad de la sesión para "probar algo". Se intentó diagnosticar un problema de AMI cambiando temporalmente a `us-east-1` (`export AWS_REGION=us-east-1`) y **no se volvió a poner `eu-central-1` antes de seguir** — el siguiente `run-instances` (paso 7) corrió contra `us-east-1` usando un `$SUBNET_ID` que es de `eu-central-1`, y tiró "**la subnet no existe**" (existe, pero no en la región contra la que se estaba consultando). Si hace falta probar algo en otra región, usar una variable aparte (`TEST_REGION`), nunca pisar `$AWS_REGION`.

Confirmá siempre que `$BACKEND_EC2_ID` sigue seteado antes de cualquier `describe-instances` que dependa de él — en esa misma sesión se perdió (sesión de CloudShell reiniciada sin volver a exportarlo, ver la sección de variables) y un `describe-instances --instance-ids "$BACKEND_EC2_ID"` corrió con la variable vacía, pisando `$SUBNET_ID` con un resultado inválido.

Si `run-instances` falla, no lo reintentas a ciegas varias veces seguidas — cada intento que sí llega a pasar la validación de parámetros (región/subnet/AMI correctos) **crea una instancia real**, aunque algo después falle. En la corrida real se corrió 4 veces seguidas mientras se diagnosticaba el problema de región/AMI — **verificar después de cualquier sesión con reintentos que no haya quedado más de una instancia:**

```
aws ec2 describe-instances --region "$AWS_REGION" \
  --filters "Name=tag:Name,Values=${PROJECT_NAME}" \
  --query 'Reservations[].Instances[].InstanceId,State.Name,SubnetId,LaunchTime,ImageId' \
  --output table
```

Si aparece más de una fila, terminar las que no estén referenciadas por la variable de entorno `INSTANCE_ID` de la función Lambda (`aws lambda get-function-configuration --function-name "${PROJECT_NAME}-daily-refresh" --query 'Environment.Variables.INSTANCE_ID'` te dice cuál es la "oficial").

Si el lookup por SSM sigue fallando después de confirmar región/versión/tipo de disco correctos, como fallback se puede fijar el AMI ID a mano (`export AMI_ID=ami-xxxxxxx`) — es lo que se terminó haciendo en la corrida real. Documentarlo así si se usa: pierde la ventaja de "siempre la última", así que conviene revisar cada tanto si el lookup dinámico ya volvió a andar.

6. user-data (instala Docker automáticamente en el primer boot)

Mismo contenido que `01-setup-ec2-ubuntu22.sh` de esta carpeta, adaptado para correr como `cloud-init`:

```
cat > /tmp/user-data.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
apt-get update
apt-get install -y ca-certificates curl gnupg
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
  > /etc/apt/sources.list.d/docker.list
apt-get update
apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
usermod -aG docker ubuntu
EOF
```

7. Lanzar la EC2

```
export INSTANCE_ID=$(aws ec2 run-instances --region "$AWS_REGION" \
  --image-id "$AMI_ID" \
  --instance-type "$INSTANCE_TYPE" \
  --subnet-id "$SUBNET_ID" \
  --security-group-ids "$NEW_SG_ID" \
  --iam-instance-profile "Name=${PROJECT_NAME}-ec2-profile" \
  --user-data file:///tmp/user-data.sh \
  --instance-initiated-shutdown-behavior stop \
  --associate-public-ip-address \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=${PROJECT_NAME}}]" \
  --query 'Instances[0].InstanceId' --output text)

echo "Instance: $INSTANCE_ID"
```

`--instance-initiated-shutdown-behavior stop` es clave: cuando el script del refresh diario termine con `shutdown -h now`, la instancia queda **detenida** (no borrada) — lista para que la Lambda la vuelva a prender al otro día.

`--associate-public-ip-address`: necesario para que el agente SSM llegue a los endpoints de AWS si la subnet no tiene NAT Gateway ni VPC endpoints de SSM — confirmá si la subnet del backend ya tiene una de esas dos cosas; si es así, se puede sacar este flag.

Esperar a que esté lista y verificar que SSM la ve:

```
aws ec2 wait instance-status-ok --region "$AWS_REGION" --instance-ids "$INSTANCE_ID"
aws ssm describe-instance-information --region "$AWS_REGION" \
  --filters "Key=InstanceIds,Values=$INSTANCE_ID"
```

⚠ En este punto la EC2 tiene Docker instalado pero **no tiene el código todavía** (`poc-pgvector-matching/` + `deploy/`) — eso se copia aparte (por ahora fuera del alcance de este manual, que es solo creación de servicios; ver `README.md` para el resto del setup).

8. Lambda — código

```

mkdir -p /tmp/lambda_build && cat > /tmp/lambda_build/handler.py <<'EOF'
import os
import boto3

INSTANCE_ID = os.environ["INSTANCE_ID"]

def handler(event, context):
    ec2 = boto3.client("ec2")
    ssm = boto3.client("ssm")

    ec2.start_instances(InstanceIds=[INSTANCE_ID])
    waiter = ec2.get_waiter("instance_status_ok")
    waiter.wait(InstanceIds=[INSTANCE_ID])

    # Fire-and-forget: no esperamos a que el refresh termine (podria pasar
    # los 15 min de limite de Lambda). El propio script apaga la EC2 al
    # terminar (shutdown -h now => se detiene, no se borra, por el
    # instance-initiated-shutdown-behavior=stop puesto al crearla).
    ssm.send_command(
        InstanceIds=[INSTANCE_ID],
        DocumentName="AWS-RunShellScript",
        Parameters={
            "commands": [
                "sudo -u ubuntu /home/ubuntu/deploy/run-daily-refresh.sh",
                "shutdown -h now",
            ]
        },
    )
    return {"status": "started", "instance_id": INSTANCE_ID}
EOF

cd /tmp/lambda_build && zip -r /tmp/lambda.zip . && cd -

```

9. Lambda — rol IAM (permisos mínimos, acotados a esta instancia)

```

cat > /tmp/lambda-trust-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{"Effect": "Allow", "Principal": {"Service": "lambda.amazonaws.com"}, "Action":
"sts:AssumeRole"}]
}
EOF

aws iam create-role --role-name "${PROJECT_NAME}-lambda-role" \
  --assume-role-policy-document file:///tmp/lambda-trust-policy.json

aws iam attach-role-policy --role-name "${PROJECT_NAME}-lambda-role" \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

# Resource acotado por PATRÓN DE NOMBRE (instance/*), no por ARN exacto - a
# propósito: este rol se crea ANTES de que la EC2 exista (paso 7 viene
# después), así que todavía no hay un INSTANCE_ID real para referenciar.
# Verificado en la corrida real del 2026-09-17: los 3 roles (pasos 4, 9, 11)
# se crean como un solo bloque, antes de lanzar nada - ver la nota debajo.
cat > /tmp/lambda-policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["ec2:StartInstances", "ec2:DescribeInstances", "ec2:DescribeInstanceStatus"],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "ssm:SendCommand",
      "Resource": [
        "arn:aws:ec2:${AWS_REGION}:${ACCOUNT_ID}:instance/*",
        "arn:aws:ssm:${AWS_REGION}::document/AWS-RunShellScript"
      ]
    }
  ]
}
EOF

aws iam put-role-policy --role-name "${PROJECT_NAME}-lambda-role" \
  --policy-name "${PROJECT_NAME}-lambda-policy" \
  --policy-document file:///tmp/lambda-policy.json

sleep 15 # IAM eventually-consistent

```

10. Lambda — crear la función

```
export LAMBDA_ROLE_ARN=$(aws iam get-role --role-name "${PROJECT_NAME}-lambda-role" \
  --query 'Role.Arn' --output text)

export LAMBDA_ARN=$(aws lambda create-function --region "$AWS_REGION" \
  --function-name "${PROJECT_NAME}-daily-refresh" \
  --runtime python3.12 \
  --role "$LAMBDA_ROLE_ARN" \
  --handler handler.handler \
  --zip-file fileb:///tmp/lambda.zip \
  --timeout 300 \
  --environment "Variables={INSTANCE_ID=$INSTANCE_ID}" \
  --query 'FunctionArn' --output text)

echo "Lambda: $LAMBDA_ARN"
```

Probar antes de programarla:

```
aws lambda invoke --region "$AWS_REGION" --function-name "${PROJECT_NAME}-daily-refresh" /tmp/lambda-
output.json
cat /tmp/lambda-output.json
```

11. EventBridge Scheduler — cron diario

```

cat > /tmp/scheduler-trust-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{"Effect": "Allow", "Principal": {"Service": "scheduler.amazonaws.com"}, "Action":
"sts:AssumeRole"}]
}
EOF

aws iam create-role --role-name "${PROJECT_NAME}-scheduler-role" \
  --assume-role-policy-document file:///tmp/scheduler-trust-policy.json

# Igual que en el rol de la Lambda: acotado por patrón de nombre
# (function:${PROJECT_NAME}-*), no por ARN exacto, porque este rol también
# se crea antes de que la función Lambda exista.
cat > /tmp/scheduler-policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [{"Effect": "Allow", "Action": "lambda:InvokeFunction", "Resource":
"arn:aws:lambda:${AWS_REGION}:${ACCOUNT_ID}:function:${PROJECT_NAME}-*"}]
}
EOF

aws iam put-role-policy --role-name "${PROJECT_NAME}-scheduler-role" \
  --policy-name "${PROJECT_NAME}-scheduler-policy" \
  --policy-document file:///tmp/scheduler-policy.json

export SCHEDULER_ROLE_ARN=$(aws iam get-role --role-name "${PROJECT_NAME}-scheduler-role" \
  --query 'Role.Arn' --output text)

sleep 15 # IAM eventually-consistent

# cron(0 2 * * ? *) = todos los días a las 02:00 UTC – ajustar horario acá
aws scheduler create-schedule --region "$AWS_REGION" \
  --name "${PROJECT_NAME}-daily-refresh" \
  --schedule-expression "cron(0 2 * * ? *)" \
  --flexible-time-window '{"Mode":"OFF"}' \
  --target "{\"Arn\":\"${LAMBDA_ARN}\",\"RoleArn\":\"${SCHEDULER_ROLE_ARN}\"}"

```

Verificar:

```
aws scheduler get-schedule --region "$AWS_REGION" --name "${PROJECT_NAME}-daily-refresh"
```

Verificación final — correr esto siempre después de una sesión con reintentos

Confirma que no quedó más de una EC2 (ver "Troubleshooting real" del paso 5 — los reintentos de `run-instances` mientras se diagnostica un error pueden dejar instancias huérfanas):

```
aws ec2 describe-instances --region "$AWS_REGION" \
  --filters "Name=tag:Name,Values=${PROJECT_NAME}" "Name=instance-state-
name,Values=pending,running,stopping,stopped" \
  --query 'Reservations[].Instances[].[InstanceId,State.Name,SubnetId,LaunchTime,ImageId]' \
  --output table
```

Debería aparecer **una sola fila**. Si hay más de una, confirmá cuál es la "oficial" (la que tiene la Lambda configurada) antes de terminar las demás:

```
aws lambda get-function-configuration --region "$AWS_REGION" \
  --function-name "${PROJECT_NAME}-daily-refresh" \
  --query 'Environment.Variables.INSTANCE_ID' --output text
```

Cualquier instancia con el mismo tag `Name` pero un `InstanceId` **distinto** al que devuelve ese comando es candidata a terminar (`aws ec2 terminate-instances --instance-ids i-xxxxx`).

Resumen de lo creado

Recurso	Nombre
Security group nuevo	<code>\${PROJECT_NAME}-sg</code>
Regla agregada al SG del backend existente	puerto <code>\$BACKEND_DB_PORT</code> desde <code>\$NEW_SG_ID</code>
Rol + instance profile EC2	<code>\${PROJECT_NAME}-ec2-role</code> / <code>\${PROJECT_NAME}-ec2-profile</code>
EC2 nueva	instance-id en <code>\$INSTANCE_ID</code> (guardalo — hace falta para todo lo que sigue)
Rol Lambda	<code>\${PROJECT_NAME}-lambda-role</code>
Función Lambda	<code>\${PROJECT_NAME}-daily-refresh</code>
Rol EventBridge Scheduler	<code>\${PROJECT_NAME}-scheduler-role</code>
Schedule	<code>\${PROJECT_NAME}-daily-refresh</code>

Para destruir todo (orden inverso, si hace falta rehacer o dar de baja)

```

aws scheduler delete-schedule --region "$AWS_REGION" --name "${PROJECT_NAME}-daily-refresh"
aws lambda delete-function --region "$AWS_REGION" --function-name "${PROJECT_NAME}-daily-refresh"
aws iam delete-role-policy --role-name "${PROJECT_NAME}-scheduler-role" --policy-name "${PROJECT_NAME}-scheduler-policy"
aws iam delete-role --role-name "${PROJECT_NAME}-scheduler-role"
aws iam delete-role-policy --role-name "${PROJECT_NAME}-lambda-role" --policy-name "${PROJECT_NAME}-lambda-policy"
aws iam detach-role-policy --role-name "${PROJECT_NAME}-lambda-role" --policy-arn
arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam delete-role --role-name "${PROJECT_NAME}-lambda-role"
aws ec2 terminate-instances --region "$AWS_REGION" --instance-ids "$INSTANCE_ID"
aws iam remove-role-from-instance-profile --instance-profile-name "${PROJECT_NAME}-ec2-profile" --role-name "${PROJECT_NAME}-ec2-role"
aws iam delete-instance-profile --instance-profile-name "${PROJECT_NAME}-ec2-profile"
aws iam detach-role-policy --role-name "${PROJECT_NAME}-ec2-role" --policy-arn
arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore
aws iam delete-role --role-name "${PROJECT_NAME}-ec2-role"
aws ec2 revoke-security-group-ingress --region "$AWS_REGION" --group-id "$BACKEND_SG_ID" --protocol tcp --port "$BACKEND_DB_PORT" --source-group "$NEW_SG_ID"
aws ec2 delete-security-group --region "$AWS_REGION" --group-id "$NEW_SG_ID"

```

Pendiente (fuera de alcance de este manual)

- Copiar `poc-pgvector-matching/` + `.env` real a la EC2 nueva y dejar corriendo `db` + `job-fetch` — la EC2 ya existe y tiene Docker instalado (user-data), pero el código todavía no está adentro. Es el próximo manual a armar, ahora que este quedó terminado.
- El disparo on-demand para usuario nuevo (Lambda/API aparte) — confirmado explícitamente fuera de alcance por ahora.
- Rotar la password del rol `matching_writer` / `matching_service` fuera de este manual (ver conversación previa sobre roles).
- Confirmar con el admin que el `iam:PutUserPolicy` / `iam:PutRolePolicy` de este manual no quedó con permisos de más de lo necesario — se usó `Resource: "*"` en algunas acciones de EC2/SSM del rol de la Lambda por simplicidad; revisar si conviene acotarlas más antes de llevar esto a producción real (hoy corre contra staging).